

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I _____CH.Bhavya Sri/192411249_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CH.Bhavya Sri

Annexure A

ERROR ANALYSIS TASK

Q1. The following program compiles successfully but produces wrong results and crashes at run time. It performs dynamic memory allocation, summation and a binary search. Analyze the logical and runtime errors in the program.

```
#include <stdio.h>
#include <stdlib.h>

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid;
        else
            high = mid;
    }
    return -1;
}

int main() {
    int *list = (int*) malloc(5 * sizeof(int));
    int i, sum;
    for(i = 0; i < 5; i++)
        list[i] = i * 10;
    for(i = 0; i <= 5; i++)
        sum += list[i];
    int pos = binarySearch(list, 5, 50);
    printf("Found at %d\n", pos);
    free(list);
    printf("%d\n", list[0]);
    int x = 10;
    if(x = 20)
        printf("x is twenty\n");
    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all logical and runtime errors
2. Mention the line number and describe the nature of each error

(b) Classification of Errors (Understanding + Analysis)

1. Separate the errors into Logical Errors and Runtime Errors
2. Justify why none of them is reported as a compile-time error

(c) Trace and Evaluation (Evaluation Level)

1. Trace `binarySearch(list, 5, 50)` for the array `{0, 10, 20, 30, 40}` and show the values of `low`, `high` and `mid` in each iteration
2. Explain why the loop never terminates and what the correct updates should be

(d) Code Correction (Application Level)

1. Rewrite the corrected program
2. Ensure that memory is accessed only while it is valid
3. Ensure the search terminates for keys that are absent

(e) Impact Analysis (Evaluation Level)

1. Explain the effect of using the uninitialized variable `sum`
2. Explain the effect of use-after-free
3. Explain the effect of using `=` instead of `==` in a condition

(f) Advanced Compiler Perspective (Create Level)

1. Suggest compiler warning options and static-analysis checks that can detect these errors
2. Describe how runtime instrumentation (bounds checking, AddressSanitizer, Valgrind) can detect them early

Q2. The following LEX specification is written to recognize identifiers, keywords, integers, floating-point numbers and operators. The specification contains several errors. Analyze it carefully.

```
%{
#include <stdio.h>
}%

digit [0-9]
letter [a-zA-Z]

%%
{letter}({letter}|{digit})* { printf("IDENTIFIER\n"); }
"if"|"else"|"while"      { printf("KEYWORD\n"); }
{digit}+                  { printf("INTEGER\n"); }
{digit}+.{digit}+         { printf("FLOAT\n"); }
"="|"+"|"-"|"*"|"|"      { printf("OPERATOR\n"); }
[ \t\n]                   { }
```

```

.                { printf("UNKNOWN\n"); }

int main() {
    yylex();
    return 0;
}

```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all errors in the LEX specification
2. Explain the effect of each error on the generated scanner

(b) Rule Matching (Understanding + Analysis)

1. Explain the two LEX conflict-resolution rules: longest match and first rule listed
2. Using these rules, explain why the input `if` is not recognized as `KEYWORD` by the given specification

(c) Token Recognition (Evaluation Level)

For the input: `if x1 = 12a4 + 3.14`

1. Show the output produced by the given specification
2. Show the output produced by the corrected specification
3. Explain why `12a4` behaves differently in the two cases

(d) Code Correction (Application Level)

1. Rewrite the corrected LEX specification
2. Add the missing section separator and the `yywrap` handling

(e) Extension (Application Level)

1. Add rules to recognize the relational operators `<`, `<=`, `==`, `!=`, `>`, `>=`
2. Add rules for a string literal and a single-line comment `//...`
3. Allow the underscore character in identifiers

(f) Advanced Thinking (Create Level)

1. Explain how LEX converts the regular expressions into a DFA and generates `lex.yy.c`
2. Write the commands to compile and run the scanner
3. Suggest how to report an error with its line number using `yylineno`

Q3. The following program fragment is passed through a lexical analyzer. It contains preprocessor directives, comments, keywords, operators and constants, and one lexical error. Analyze how the scanner processes the program.

```

#include <stdio.h>
#define LIMIT 100

int main() {
    int total = 0, i;
    float rate = 2.5e+;
    /* sum of first n numbers */
    for(i = 1; i <= LIMIT; i++) {

```

```

        total += i; // accumulate
    }
    if(total != 5050 && rate > 2) {
        printf("Sum = %d\n", total);
    }
    return 0;
}

```

Tasks:

(a) Token Identification (Analysis Level)

1. List the lexemes of the statement `for(i = 1; i <= LIMIT; i++) {` and give the token class of each (keyword, identifier, constant, operator, punctuation)
2. Count the total number of tokens in that statement
3. Count the tokens in the statement `total += i;`

(b) Lexical Error Analysis (Analysis Level)

1. Identify the lexical error and its line number
2. Explain which rule of the numeric constant is violated and suggest a valid replacement

(c) Scanner Behavior (Understanding Level)

1. State which parts of the program (comments, whitespace, preprocessor lines) are discarded and which are passed to the parser
2. Explain how the scanner distinguishes `+`, `++` and `+=` and also `<`, `<=` using longest match

(d) Symbol Table (Application Level)

1. Construct the symbol table entries for the identifiers `total`, `i`, `rate` and `LIMIT` with attributes: name, token class, data type, scope
2. Explain which phase fills in the data type attribute

(e) Regular Expressions (Application Level)

1. Write regular definitions for: integer constant, floating-point constant with exponent, identifier and relational operators
2. Show how the definitions accept `5050` and reject `2.5e+`

(f) Advanced Task (Create Level)

1. Explain the two-buffer input scheme with sentinels and how it supports look-ahead
2. Suggest improvements to the scanner for better error recovery and error messages

① Error Analysis

a) Error Identification

The program has the following errors. The line numbers are from the given program

Line	Error	Type	Explanation
26	high = n	logical	Last valid index is n-1, not n
27	while (low <= high)	logical	With the given updates, the search can get stuck
32	low = mid	logical	Should be low = mid + 1
34	high = mid	logical	Should be high = mid - 1
40	int i, sum;	Runtime/ logical	sum is uninitialized
43	i <= 5	Runtime	Accesses list[5], which is outside allocated memory
44	sum += list[i]	Runtime/ logical	Uses uninitialized sum and may access list[5]
48	list[0] after free()	Runtime	Use-after-free
50	if (x = 20)	logical	Assignment is used instead of Comparison.

b) Classifications

logical errors:

1. $high = n$
2. $low = mid$
3. $high = mid$
4. Uninitialized sum
5. $if (x = 20)$

Runtime errors:

1. $list[5]$ out-of-bounds access.
2. Accessing $list[0]$ after $free(list)$.

c) Binary Search Trace

Array: $\{0, 10, 20, 30, 40\}$

key = 50

Given Code: $low = 0$
 $high = 5$

Iteration 1

$low = 0$

$high = 5$

$mid = (0 + 5) / 2 = 2$

$arr[2] = 20$

$20 < 50, 50 >$

$$\text{low} = \text{mid} = 2$$

Iteration 2

$$\text{low} = 2$$

$$\text{high} = 5$$

$$\text{mid} = (2+5)/2 = 3$$

$$\text{arr}[3] = 30$$

$$30 < 50;$$

$$\text{low} = 3$$

Iteration 3

$$\text{low} = 3$$

$$\text{high} = 5$$

$$\text{mid} = (3+5)/2 = 4$$

$$\text{arr}[4] = 40$$

$$40 < 50;$$

$$\text{low} = 4$$

So, the values remain

$$\text{low} = 4$$

$$\text{high} = 5$$

$$\text{mid} = 4$$

forever.

Correct updates

$$\text{high} = \text{high} - 1; \text{ and } \text{low} = \text{mid} + 1; \text{ and}$$

$$\text{high} = \text{mid} - 1;$$

1. INCORRECT CODE:

The screenshot shows the Programiz Online C Compiler interface. The code in the editor is as follows:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int binarySearch(int arr[], int n, int key)
5 {
6     int low = 0, high = n;
7
8     while (low <= high)
9     {
10         int mid = (low + high) / 2;
11
12         if (arr[mid] == key)
13             return mid;
14         else if (arr[mid] < key)
15             low = mid;
16         else
17             high = mid;
18     }
19
20     return -1;
21 }
22
```

The output window displays the message: `=== Session Ended Due To Inactivity ===`

CORRECT CODE:

The screenshot shows the Programiz Online C Compiler interface with the corrected code. The code in the editor is as follows:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int binarySearch(int arr[], int n, int key)
5 {
6     int low = 0, high = n - 1;
7
8     while (low <= high)
9     {
10         int mid = (low + high) / 2;
11
12         if (arr[mid] == key)
13             return mid;
14         else if (arr[mid] < key)
15             low = mid + 1;
16         else
17             high = mid - 1;
18     }
19
20     return -1;
21 }
22
```

The output window displays the message: `Sum = 100`
`Not found`

```

printf("Found at %d\n", binarySearch(list, 50));
free(list);
return 0;

```

e) Effects

- Uninitialized sum → gives unpredictable result.
- Use-after-free → Undefined behavior / crash
- `=` instead of `==` → assigns 20 to x, Condition becomes true

f) Detection

gcc -Wall -Wextra
 -cppcheck
 clang-tidy
 AddressSanitizer
 Valgrind.

② LEX

A: a) Errors

1. Keyword rule is after identifier rule
2. `.in` float rule should be `1..`
3. Missing `%%` before `main()`.
4. Need `yywrap()` or `%o` option `noyywrap`

b) LEX Rules

Longest match: longest possible token is selected.

First rule's if lengths are equal, first is selected

so it becomes IDENTIFIER because identifier rules appears first

c) Input:

if $x_1 = 12a_4 + 3.14$

Given:

IDENTIFIER

IDENTIFIER

OPERATOR

INTEGER

IDENTIFIER

OPERATOR

FLDAT

Corrected:

KEYWORD

IDENTIFIER

OPERATOR

INTEGER

IDENTIFIER

OPERATOR

FLOAT

d) Corrected rules

% option keyword

%

"if" | "else" | "while" { printf("KEYWORD\n"); }

[a-zA-Z][a-zA-Z0-9]*

{ printf("IDENTIFIER\n"); }

[0-9]+ | [0-9]+ { printf("FLOAT\n"); }

[0-9]+ { printf("INTEGER\n"); }

e) Advanced

LEX: Regular Expression $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ lex.yy.c

Commands: lex scanner.l gcc lex.yy.c -o scanner
./scanner

2. INCOORECT CODE:

```
lex.l - Notepad
File Edit Format View Help
%{
#include <stdio.h>
%}

digit [0-9]
letter [a-zA-Z]

%%

"if"|"else"|"while"      { printf("KEYWORD\n"); }
{digit}+"."{digit}+      { printf("FLOAT\n"); }
{digit}+                  { printf("INTEGER\n"); }
{letter}{letter}{digit}* { printf("IDENTIFIER\n"); }
"="|"+"|"-"|"*"|"/"      { printf("OPERATOR\n"); }
[ \t\n]+                  { }
.                          { printf("UNKNOWN\n"); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    printf("Enter input:\n");
    yylex();
    return 0;
}
```

CORRECT CODE:

Online Compiler

Flex (Lex)

Run

Save

Share

scanner.l

```
1  %{
2  #include <stdio.h>
3  %}
4
5  digit  [0-9]
6  letter [a-zA-Z]
7
8  %%
9  "if"|"else"|"while"      { printf("KEYWORD\n"); }
10 {digit}+"."{digit}+      { printf("FLOAT\n"); }
11 {digit}+                  { printf("INTEGER\n"); }
12 {letter}{letter}{digit}* { printf("IDENTIFIER\n"); }
13 "="|"+"|"-"|"*"|"/"      { printf("OPERATOR\n"); }
14 [ \t\n]+                  { }
15 .                          { printf("UNKNOWN\n"); }
16
17 %%
18
19 int yywrap()
20 {
21     return 1;
22 }
23
24
25 int main()
26 {
27     printf("Enter input:\n");
28     yylex();
29     return 0;
30 }
```

Input

```
if x1 = 12a4 + 3.14
```

Output

```
Enter input:
if x1 = 12a4 + 3.14
KEYWORD
IDENTIFIER
OPERATOR
INTEGER
IDENTIFIER
OPERATOR
FLOAT

Process exited with code 0
```

Lexical Analysis

a) Tokens

for(i=1; i<=LIMIT; i++) {

for $\rightarrow$ keyword

($\rightarrow$ punctuation

i $\rightarrow$ Identifier

= $\rightarrow$ operator

1 $\rightarrow$ Constant

; $\rightarrow$ punctuation

i $\rightarrow$ Identifier

<= $\rightarrow$ operator

LIMIT $\rightarrow$ Identifier

; $\rightarrow$ punctuation

i $\rightarrow$ Identifier

++ $\rightarrow$ operator

) $\rightarrow$ punctuation

{ $\rightarrow$ punctuation

Total = 14 tokens

total += 1; $\rightarrow$ 4 tokens

b) lexical error

float rate = 2.5 et;

Error: 2.5et

A digit is required after et.

Correct: 2.5et2

c) Scanner

- Whitespace - discarded
- Comments → discarded
- Preprocessor lines → handled separately
- longest match recognizes $++$, $+=$, $<=$

d) Symbol Table

Name	Type	Scope
total	int	Local
i	int	Local
rate	float	Local
LIMIT	Macro	File

Data type → Semantic Analysis

e) Regular Expressions

Integer: $[0-9]^+$

Float: $[0-9]^+ \cdot [0-9]^+ ([eE][+-]? [0-9]^+)^?$

Identifier: $[a-zA-Z][a-zA-Z0-9]^*$

Relational: $<= | >= | == | != | < | >$

5050 → accepted

2.5e+ → rejected

f) Two-Buffer Scheme

Two buffers store input alternately with sentinels. It provides:

- look-ahead
- faster input processing
- fewer input operations.

For error reporting
Use:

%option yylineno

3..INCORRECT CODE:

The screenshot shows the Programiz C Online Compiler interface. The code in the editor is as follows:

```
1 %{\n2 #include <stdio.h>\n3 %}\n4\n5 digit [0-9]\n6 letter [a-zA-Z]\n7\n8 %%\n9 {letter}({letter}|{digit})* { printf("IDENTIFIER\\n"); }\n10 "if"|"else"|"while" { printf("KEYWORD\\n"); }\n11 {digit}+ { printf("INTEGER\\n"); }\n12 {digit}+.{digit}+ { printf("FLOAT\\n"); }\n13 "=="|"!="|"=="|"!="|"=="|"!=" { printf("OPERATOR\\n"); }\n14 [ \\t\\n ] {\n15     . { printf("UNKNOWN\\n"); }\n16 }\n17\n18 int main() {\n19     yylex();\n20     return 0;\n21 }
```

The output window displays several compiler errors:

```
ERROR!\n/tmp/LSWirtKiz-B03I1rnm/main.c:1:1: error: expected identifier or '('\nbefore '%' token\n1 | %{\n  | ^\n\nERROR!\n/tmp/LSWirtKiz-B03I1rnm/main.c:6:1: error: expected '=', ',', ';', 'asm'\nor '__attribute__' before 'letter'\n6 | letter [a-zA-Z]\n  | ^~~~~~\n\n/tmp/LSWirtKiz-B03I1rnm/main.c:9:10: error: expected identifier or '('\nbefore '{' token\n9 | {letter}({letter}|{digit})* { printf("IDENTIFIER\\n"); }\n  | ^\n\n/tmp/LSWirtKiz-B03I1rnm/main.c:10:1: error: expected identifier or '('\nbefore string constant\n10 | "if"|"else"|"while" { printf("KEYWORD\\n"); }\n   | ^~~~~\n\n/tmp/LSWirtKiz-B03I1rnm/main.c:11:1: error: expected identifier or '('\nbefore '{' token\n11 | {digit}+ { printf("INTEGER\\n"); }\n   | ^
```

CORRECT CODE:

The screenshot shows the Programiz C Online Compiler interface with the following code:

```
1 #include <stdio.h>\n2\n3 #define LIMIT 100\n4\n5 int main()\n6 {\n7     int total = 0, i;\n8     float rate = 2.5e+2;\n9\n10    for(i = 1; i <= LIMIT; i++)\n11    {\n12        total += i;\n13    }\n14\n15    if(total == 5050 && rate > 2)\n16    {\n17        printf("Sum = %d\\n", total);\n18        printf("Rate = %.2f\\n", rate);\n19    }\n20\n21    return 0;\n22 }
```

The output window shows the successful execution results:

```
Sum = 5050\nRate = 250.00
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand